

Aegis-Lock

A Privacy-First End-to-End Encrypted Password Manager Extension

A Project Report Submitted by

Krish Mahajan - 102303527

Aryan Ranjan - 102303528

Siddhant Kapoor - 102303529

Shantanu Singhal - 102353013

Thapar Institute of Engineering and Technology

Abstract

In an era of increasing digital dependency, the security of user credentials has become paramount. This project, **Aegis-Lock**, introduces a browser-based password manager built on a "Zero-Knowledge" architecture. Unlike traditional managers that may store keys server-side, Aegis-Lock performs all cryptographic operations—specifically AES-256-GCM encryption—client-side within a Chrome Extension environment. The system utilizes a React-based frontend for credential management and a NodeJS/MongoDB backend deployed on Vercel for encrypted storage. This report details the necessity of such a system, analyzes its technical and economic feasibility, and evaluates its performance regarding user experience and security integrity.

Contents

Abstract	1
1 Introduction & Project Need	2
1.1 Overview	2
1.2 The Need for Aegis-Lock	2
2 Feasibility Study	3
2.1 Technical Feasibility	3
2.2 Economic Feasibility	3
2.3 Operational Feasibility	4
3 System Architecture & Implementation	5
3.1 Technology Stack	5
3.2 Security Architecture	5
3.3 Key Features Implemented	6
4 User Evaluation	7
4.1 Performance Testing	7
4.2 Security Validation	7
5 Conclusion & Future Scope	8
5.1 Conclusion	8
5.2 Future Scope	8

Chapter 1

Introduction & Project Need

1.1 Overview

Aegis-Lock is a cross-platform password manager implemented as a browser extension. It integrates directly into the user's browsing experience to detect login forms, autosave new credentials, and autofill existing ones securely. The core philosophy is that the server is "blind"—it stores data it cannot read.

1.2 The Need for Aegis-Lock

Despite the prevalence of cybersecurity warnings, password hygiene remains poor among the general public. The development of Aegis-Lock addresses several critical market needs:

1. **Mitigation of Password Reuse:** Users tend to reuse passwords across multiple sites. A breach in one service can compromise a user's entire digital identity. Aegis-Lock removes the burden of memorization, encouraging unique, complex passwords.
2. **Phishing Protection:** The autofill mechanism checks the exact domain (`window.location.hostname`) before filling credentials. This protects users from phishing sites that visually mimic legitimate services but have different domains.
3. **Data Sovereignty:** Many commercial password managers are closed-source. Aegis-Lock provides an open-source, auditable alternative where users can verify that their Master Password is never transmitted over the network.
4. **Seamless Integration:** Desktop-based password managers often require context switching. A browser extension reduces friction by injecting controls directly into the DOM of the login page.

Chapter 2

Feasibility Study

Before development, a rigorous analysis was conducted to determine the viability of the project.

2.1 Technical Feasibility

The project relies on established web standards, ensuring high technical feasibility:

- **Cryptography:** The Web Crypto API (`subtleCrypto`) is natively supported in all modern browsers, allowing for high-performance AES-GCM and PBKDF2 operations without external heavy libraries.
- **Extension Architecture:** Chrome's Manifest V3 platform provides the necessary APIs (`'chrome.storage'`, `'chrome.scripting'`) to handle secure storage and page interaction.
- **Scalability:** The backend uses MongoDB Atlas, a NoSQL database that scales horizontally. Since the server performs no heavy cryptographic computation (only storage), it can handle high concurrency with minimal resources.

2.2 Economic Feasibility

The project is highly cost-effective:

- **Development Costs:** Built using open-source technologies (React, Vite, Node.js), requiring no licensing fees.
- **Deployment Costs:** The backend API is deployed on **Vercel**, a serverless platform that offers a generous free tier and automatic scaling. Combined with a free-tier MongoDB Atlas instance, the operating cost is near zero for a moderate user base.

2.3 Operational Feasibility

The system requires no special training. The user interface mimics standard login flows (popup entry, click-to-fill), ensuring a shallow learning curve. The "Zero-Knowledge" model simplifies legal compliance (GDPR), as the service provider strictly cannot access user data.

Chapter 3

System Architecture & Implementation

3.1 Technology Stack

Component	Technology
Frontend Extension	React 18, Vite, Chrome Manifest V3
Backend API	Next.js (Node.js)
Deployment	Vercel (Serverless Functions)
Database	MongoDB Atlas
Encryption	Web Crypto API (AES-256-GCM, PBKDF2)
Styling	CSS3, Flexbox

Table 3.1: Technology Stack

3.2 Security Architecture

The security model is the cornerstone of Aegis-Lock:

1. **Key Derivation:** A Master Key is derived from the user's password using PBKDF2 (300,000 iterations) with a unique random salt.
2. **Double Encryption:** The Vault is encrypted with a random 256-bit Encryption Key. This Encryption Key is itself encrypted by the Master Key. This allows the user to change their password without re-encrypting the entire vault.
3. **Transport Security:** All data is transmitted over HTTPS via Vercel's secure edge network. Authentication uses JWT (JSON Web Tokens) derived from a hash of the Master Key, ensuring the server never sees the raw password.

3.3 Key Features Implemented

- **Smart Autosave:** A 'MutationObserver' on the content script detects dynamic form submissions (common in Single Page Applications) and prompts the user to save credentials via a banner.
- **Non-Intrusive Autofill:** Instead of automatically filling fields (which can be a privacy risk), Aegis-Lock injects a discrete "Key Icon" into login fields. Users must click this icon to fill credentials.
- **Secure Session Management:** Encryption keys are stored in 'sessionStorage' and backed up to 'chrome.storage.local' to persist across popup closures, but are strictly cleared upon logout.

Chapter 4

User Evaluation

The system was evaluated based on performance, usability, and security integrity.

4.1 Performance Testing

- **Encryption Speed:** The Web Crypto API encrypts a standard vault (50+ passwords) in under 15ms, providing a perceptible "instant" save.
- **Page Load Impact:** The content script is lightweight (< 20KB) and uses 'MutationObserver' rather than polling, ensuring zero impact on webpage rendering performance (Lighthouse score unaffected).
- **API Latency:** Deployed on Vercel's edge network, API response times averaged under 100ms for vault retrieval operations.

4.2 Security Validation

- **Network Sniffing:** Packet inspection confirmed that payloads sent to '/api/vault/update' are fully opaque Base64 strings. No plaintext data leaks.
- **Database Breach Simulation:** Accessing the raw MongoDB collection revealed only 'encryptedVault' blobs. Without the client-side key, recovery was mathematically impossible.

Chapter 5

Conclusion & Future Scope

5.1 Conclusion

Aegis-Lock successfully demonstrates that high-security, zero-knowledge architecture can be delivered with a consumer-grade user experience. By leveraging modern browser APIs and a robust React frontend deployed on reliable infrastructure like Vercel, the project satisfies the critical need for secure password management without relying on expensive proprietary software. The system is technically feasible, economically viable, and performs robustly in real-world scenarios.

5.2 Future Scope

- **Biometric Unlock:** Integration with Windows Hello / TouchID via WebAuthn.
- **Password Health Check:** Client-side analysis of password strength and reuse detection.
- **Cross-Device Sync:** Developing a React Native mobile application that syncs with the same MongoDB backend.